

第五讲 (续) 基于 “集体智慧” 的预测算法 ---集成学习(续)

主要内容

- 继续讨论案例二的回归预测问题
 - 组合预测进阶：集成学习
 - 再看弱模型到强模型的构建
 - 梯度提升算法和梯度提升树
 - 超参数调优策略
 - 梯度提升树的XGBoost算法

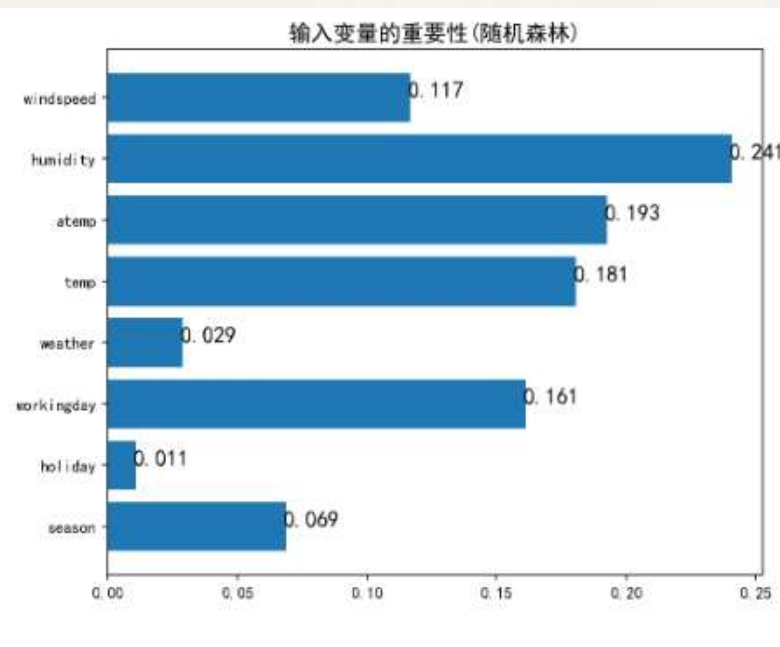
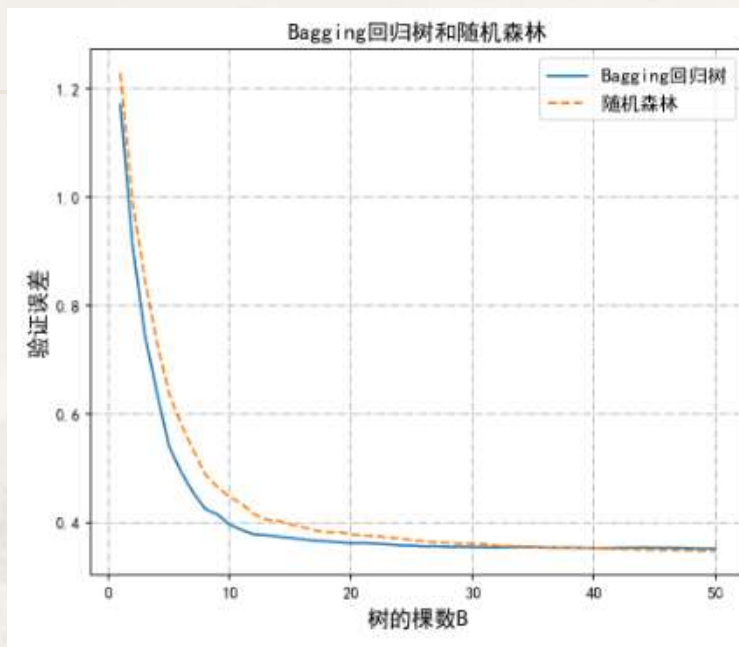
案例二：共享单车日租赁量的预测：回归预测

- 案例二：共享单车日租赁量 (Casual/Registered/count) 的预测：回归预测

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- Season – 1:spring 2:summer 3:fall 4:winter
- Holiday – whether the day is considered a holiday
- Workingday: whether the day is neither a weekend nor holiday
- Weather -
 - 1: Clear, Few clouds, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- Temp - temperature in Celsius
- Atemp - "feels like" temperature in Celsius
- Humidity - relative humidity
- Windspeed - wind speed
- Casual – number of non-registered user rentals initiated
- Registered - number of registered user rentals initiated
- count - number of total rentals

- 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林



```
dtrTestErr = mean_absolute_error(y_test,modelDTC.predict(X_test))
bagTestErr = mean_absolute_error(y_test,Bag.predict(X_test))
rfTestErr = mean_absolute_error(y_test,RF.predict(X_test))
print(f'测试误差(MAE): \n    单课决策树={dtrTestErr:.3f}\n    Bagging={bagTestErr:.3f}\n    随机森林={rfTestErr:.3f}')
```

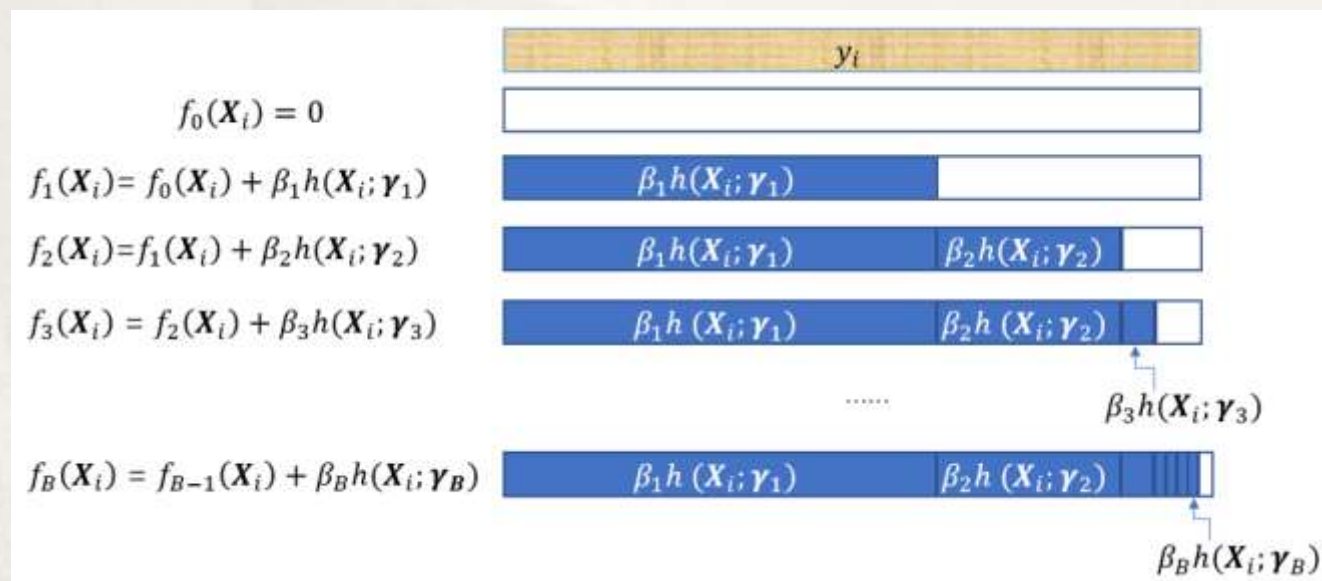
测试误差(MAE):
单课决策树=19.961
Bagging=18.928
随机森林=18.011

- 问：从集成学习角度看，误差还有进一步降低的空间吗？
 - 让各基学习器聚焦不同的局部数据并进行联合预测？
 - 再看从弱模型到强模型的向前式分步可加建模策略
 - 梯度提升算法和梯度提升树
 - 梯度提升树的XGBoost、LightGBM算法，等

- 问：从集成预测角度看，误差还有进一步降低的空间吗？
 - 再看集成学习：从弱模型到强模型的向前式分步可加建模策略

$$f_b(X) = f_{b-1}(X) + \beta_b G_b(X; \gamma_b) \quad \beta_b \text{ 为模型系数}$$

- 将该思路拓展：
 - 迭代开始前，令当前预测值 $f_0(X) = 0$
 - 开始 $b = 1, \dots, B$ 次的迭代
 - $f_1(X) = f_0(X) + \beta_1 h(X; \gamma_1)$,
 - $f_2(X) = f_1(X) + \beta_2 h(X; \gamma_2)$,
 - $f_3(X) = f_2(X) + \beta_3 h(X; \gamma_3)$
 - $f_b(X) = f_{b-1}(X) + \beta_b h(X; \gamma_b)$
 - B 次迭代结束时的预测值为：
 - $f_B(X) = \sum_{b=1}^B \beta_b h(X; \gamma_b)$
 - 回归预测中的梯度提升算法



- 问：
 - 从预测角度看，回归预测中的 $h(X; \gamma_b)$ 做了什么？什么是弱模型？
 - 修正预测值(提升)；对 $b-1$ 次迭代后残差的拟合
 - 可否找到 $b-1$ 次迭代后残差的统一表达形式？

- 问：可否找到 **$b-1$ 次迭代后残差的统一表达形式？**

$$f_b(\mathbf{X}) = f_{b-1}(\mathbf{X}) + \beta_b h(\mathbf{X}; \boldsymbol{\gamma}_b)$$

- 回归预测中的**损失函数**： $L(y_i, f(\mathbf{X}_i)) = \frac{1}{2} (y_i - f(\mathbf{X}_i))^2$

$$-g_b(\mathbf{X}_i) = - \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

- $\tilde{y}_i = -g_b(\mathbf{X}_i)$ ， $\tilde{y}_i = y_i - f_{b-1}(\mathbf{X}_i)$ ，为 **$b-1$ 次迭代后的残差**，称 $\tilde{y}_i$ 为**伪响应变量**

- 回归预测中的梯度提升算法步骤：

- 迭代开始前，令当前预测值 $f_0(\mathbf{X}) = \bar{y}$ 。进行 $b = 1, \dots, B$ 次如下迭代：

- 第一步，计算伪响应变量：

$$\tilde{y}_i = y_i - f_{b-1}(\mathbf{X}_i), i = 1, 2, \dots, N$$

- 第二步，求解当前模型的参数：

$$(\boldsymbol{\gamma}_b, \beta_b) = \arg \min_{\boldsymbol{\gamma}, \beta} \sum_{i=1}^N (\tilde{y}_i - \beta h(\mathbf{X}_i; \boldsymbol{\gamma}))^2$$

找到伪响应变量(残差)拟合最好下的回归树--- GBDT (Gradient Boosting Decision Tree)

- 第三步，更新：

$$f_b(\mathbf{X}) = f_{b-1}(\mathbf{X}) + \beta_b h(\mathbf{X}; \boldsymbol{\gamma}_b)$$

- 问：为何命名为**梯度提升**？如何看待 β_b ？

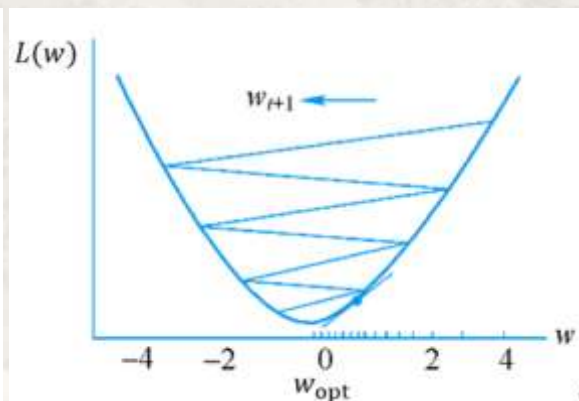
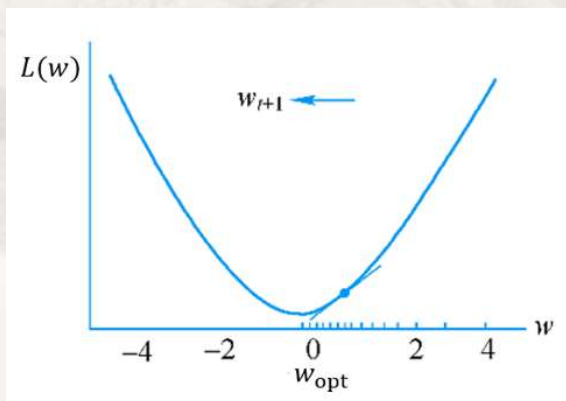
- 问：为何命名为梯度提升？如何看待 β_b ？

- 残差通过计算梯度得到

- 再看梯度下降法的参数优化

- 有损失函数 $L(w) = w^2 + 1$ ，求解 $L(w)$ 最小时的 w 的值
- 多次迭代，计算 $L(w)$ 在 w_{b-1} 处的导数并不断更新 w

$$w_b = w_{b-1} + \Delta w = w_{b-1} - \rho \left[\frac{\partial L(w)}{\partial w} \right]_{w=w_{b-1}}$$



ρ 称为学习率，会影响 Δw ，不可太大或太小，一般为可变的数值

```
1  ##简单的梯度下降法
2  w = 4
3  rho = 0.1
4  for i in np.arange(1,20):
5      w = w - rho * (2 * w)
6      print('第%d次迭代:w= %.5f,函数值= %.5f'%(i,w,w**2+1))
```

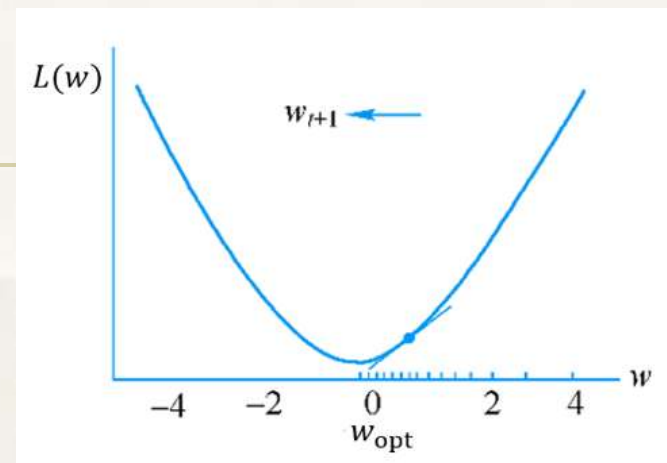
第1次迭代: w=3.20000, 函数值=11.24000
第2次迭代: w=2.56000, 函数值=7.55360
第3次迭代: w=2.04800, 函数值=5.19430
第4次迭代: w=1.63840, 函数值=3.68435
第5次迭代: w=1.31072, 函数值=2.71799
第6次迭代: w=1.04858, 函数值=2.09951
第7次迭代: w=0.83886, 函数值=1.70369
第8次迭代: w=0.67109, 函数值=1.45036
第9次迭代: w=0.53687, 函数值=1.28823
第10次迭代: w=0.42950, 函数值=1.18447
第11次迭代: w=0.34360, 函数值=1.11806
第12次迭代: w=0.27488, 函数值=1.07556
第13次迭代: w=0.21990, 函数值=1.04836
第14次迭代: w=0.17592, 函数值=1.03095
第15次迭代: w=0.14074, 函数值=1.01981
第16次迭代: w=0.11259, 函数值=1.01268
第17次迭代: w=0.09007, 函数值=1.00811
第18次迭代: w=0.07206, 函数值=1.00519
第19次迭代: w=0.05765, 函数值=1.00332

• 问：为何命名为梯度提升？如何看待 β_b ？

• 再看梯度下降法的参数优化

• 多次迭代，计算 $L(w)$ 在 w_{b-1} 处的导数并不断更新 w

$$w_b = w_{b-1} + \Delta w = w_{b-1} - \rho \left[\frac{\partial L(w)}{\partial w} \right]_{w=w_{b-1}}$$



• 梯度提升算法中： $f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) +$ 修正预测值 $\beta_b h(\mathbf{X}_i; \gamma_b)$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \Delta f(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) - \rho_b \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

• $w_b \rightarrow f_b(\mathbf{X}_i)$ ，损失函数 $L(y_i, f(\mathbf{X}_i))$ ，依梯度下降法思路进行模型优化

$$(\gamma_b, \beta_b) = \arg \min_{\gamma, \beta} \sum_{i=1}^N (\tilde{y}_i - \beta h(\mathbf{X}_i; \gamma))^2$$

学习率

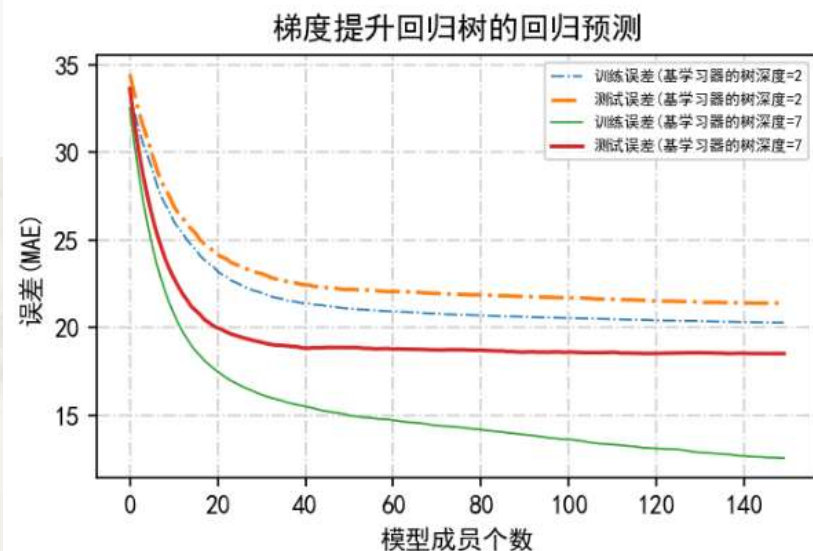
• 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林→GBDT

```
1 X = data[['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed']]
2 y = data['casual']
3 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
4 B = 150
5 fig = plt.figure(figsize = (5, 3), dpi = 150)
6 for d, lty in zip([2, 7], ['-.', '-']):
7     GBRT = ensemble.GradientBoostingRegressor(loss = 'ls', n_estimators = B, max_depth = d, random_state = 123)
8     GBRT.fit(X_train, y_train)
9     ErrTrainGBRT = np.zeros((B,)); ErrTestGBRT = np.zeros((B,))
10    for b, yhat in enumerate(GBRT.staged_predict(X_train)):
11        ErrTrainGBRT[b] = mean_absolute_error(y_train, yhat)
12    for b, yhat in enumerate(GBRT.staged_predict(X_test)):
13        ErrTestGBRT[b] = mean_absolute_error(y_test, yhat)
14    plt.plot(np.arange(B), ErrTrainGBRT, linestyle = lty, label = f'训练误差(基学习器的树深度={d}', linewidth = 0.8)
15    plt.plot(np.arange(B), ErrTestGBRT, linestyle = lty, label = f'测试误差(基学习器的树深度={d}')
16    GBRTTestErr = mean_absolute_error(y_test, GBRT.predict(X_test))
17    print(f'测试误差(MAE): GBDT(基学习器的最大树深度={d})={GBRTTestErr:.3f}')
18 plt.title('梯度提升回归树的回归预测', fontsize = 12)
19 plt.grid(True, linestyle = '-.', alpha = 0.6)
20 plt.xlabel('模型成员个数', fontsize = 10)
21 plt.ylabel('误差(MAE)', fontsize = 10)
22 plt.legend(fontsize = 6)
```

测试误差(MAE): GBDT(基学习器的最大树深度=2)=21.407
测试误差(MAE): GBDT(基学习器的最大树深度=7)=18.528

测试误差(MAE):
单棵决策树=19.961
Bagging=18.928
随机森林=18.011

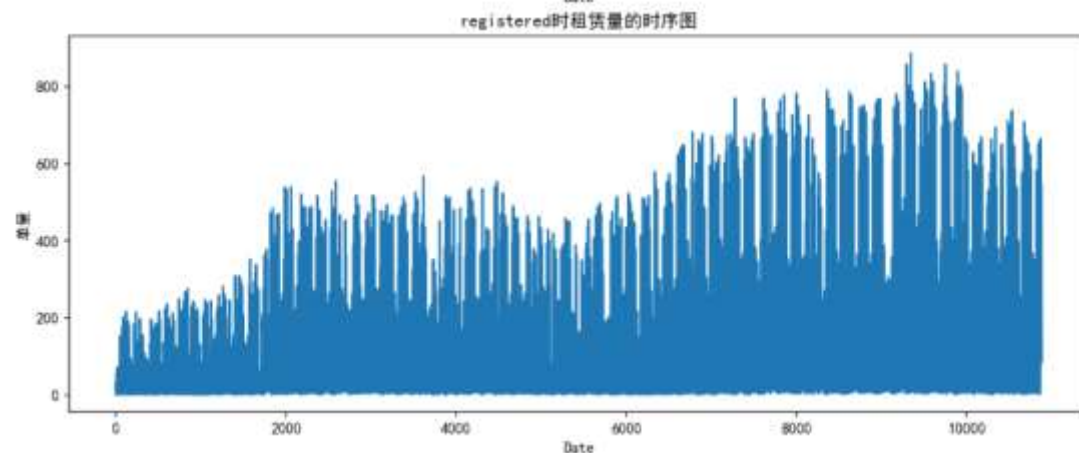
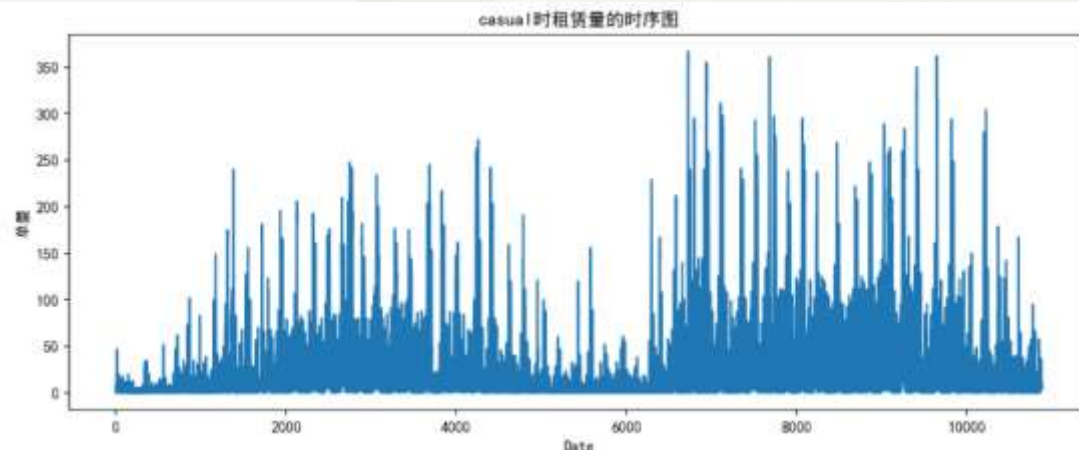
- 可尝试通过调参进一步优化模型
 - 树模型的参数、迭代次数, 等
- 案例数据的再观察(邵佳乐)



- 案例二：数据的再观察：2011年1月至2021年12月，每月20日前的小时单量数据

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0000	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0000	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0000	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0000			
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0000			
5	2011-01-01 05:00:00	1	0	0	2	9.84	12.880	75	6.0032			
6	2011-01-01 06:00:00	1	0	0	1	9.02	13.635	80	0.0000			
7	2011-01-01 07:00:00	1	0	0	1	8.20	12.880	86	0.0000			
8	2011-01-01 08:00:00	1	0	0	1	9.84	14.395	75	0.0000			
9	2011-01-01 09:00:00	1	0	0	1	13.12	17.425	76	0.0000			

```
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(12, 10), dpi=150)
for l, Y in zip([0, 1], ['casual', 'registered']):
    y = data[Y] #registered, casual
    axes[l].plot(y)
    axes[l].set_title(f'{Y}时租赁量的时序图')
    axes[l].set_xlabel('Date')
    axes[l].set_ylabel('单量')
```



- 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林→GBDT→基于时间序列的GBDT

- 基于小时数据，回看6小时进行提前1小时的预测

$$\hat{y}_{t+1} = \delta_0 + \beta_1 y_t + \cdots + \beta_6 y_{t-5} + \alpha_1 x_t + \cdots + \alpha_6 x_{t-5}$$

- 可采用经典时间序列方法(ARMA, ARIMA, 等)
- 采用GBDT算法

- 案例二的分析： **基于时间序列的GBDT**
 - 数据**预处理**：计算生成新的输入变量

$$\hat{y}_{t+1} = \delta_0 + \beta_1 y_t + \cdots + \beta_6 y_{t-5} + \alpha_1 x_t + \cdots + \alpha_6 x_{t-5}$$

提前1小时预测(回看6小时)

```

1 y = 'casual'
2 df = data.copy()
3 delay = 1                                #目标在未来多少个时间步之后
4 lookback = 6                             #回看6个时间步
5 varList = [y, 'weather', 'temp', 'atemp', 'humidity', 'windspeed'] # 'season', 'holiday', 'workingday'
6 for i, name in enumerate(varList, start = 1):
7     for j in np.arange(lookback-1):
8         newname = name + 't-%d'%(delay + j)
9         df[newname] = df[name].shift(delay + j) #滞后delay+j个时间步
10 print(df.columns)

```

```

Index(['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp',
      'atemp', 'humidity', 'windspeed', 'casual', 'registered', 'count',
      'date_time', 'casualt-1', 'casualt-2', 'casualt-3', 'casualt-4',
      'casualt-5', 'weathert-1', 'weathert-2', 'weathert-3', 'weathert-4',
      'weathert-5', 'tempt-1', 'tempt-2', 'tempt-3', 'tempt-4', 'tempt-5',
      'atempt-1', 'atempt-2', 'atempt-3', 'atempt-4', 'atempt-5',
      'humidityt-1', 'humidityt-2', 'humidityt-3', 'humidityt-4',
      'humidityt-5', 'windspeedt-1', 'windspeedt-2', 'windspeedt-3',
      'windspeedt-4', 'windspeedt-5'],
      dtype='object')

```

```

1 print(df.shape)
2 df.dropna(inplace = True) #删除不完整观测
3 print(df.shape)

```

(10886, 43)

(10881, 43)

• 案例二的分析： 基于时间序列的GBDT

• 数据预处理：数据集划分

```
data['date_time'] = pd.to_datetime(data['datetime']) # 转换为 pandas 的日期格式
```

```
1 # 计算总天数，留20%做测试集
2 num_days = (df['date_time'].max() - df['date_time'].min()).days
3 OOB = num_days*0.2
4 # 计算训练集和测试集的分隔时间
5 cutdate = df['date_time'].max() - pd.Timedelta(days = OOB)
6
7 # 指定训练集
8 train = df.loc[(df['date_time'] <= cutdate), :]
9 print('Training data: from', train['date_time'].min(), 'to', train['date_time'].max() )
10 # 指定测试集
11 test = df.loc[(df['date_time'] > cutdate), :]
12 print('Test data: from', test['date_time'].min(), 'to', test['date_time'].max() )
```

Training data: from 2011-01-01 06:00:00 to 2012-07-19 23:00:00

Test data: from 2012-08-01 00:00:00 to 2012-12-19 23:00:00

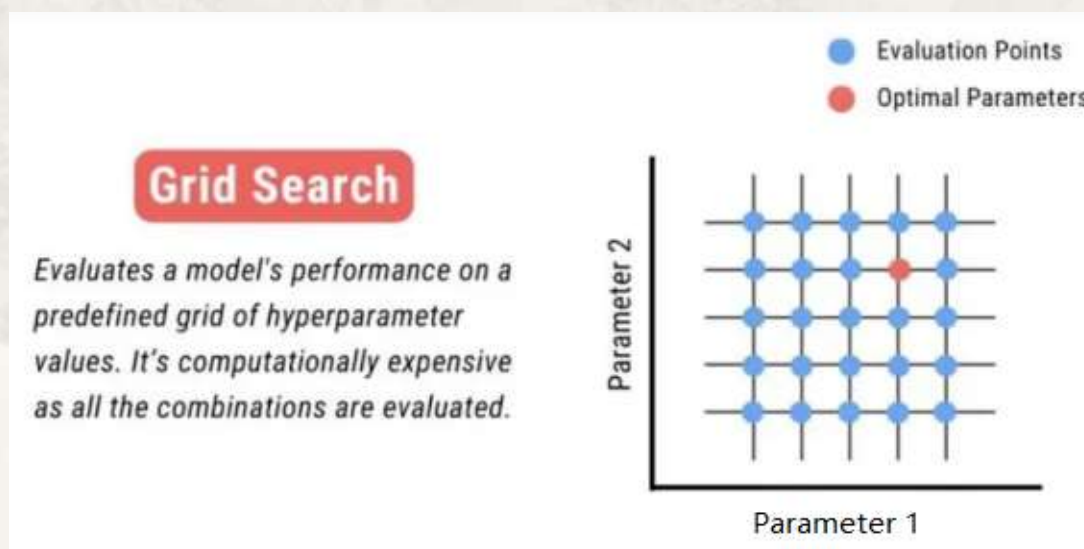
• 指定输出变量和输入变量

```
1 y_train = train[y]; y_test = test[y]
2 cols1 = df.columns[1:9].values.tolist(); cols2 = df.columns[13:].values.tolist()
3 cols = cols1 + cols2
4 print(cols)
5 X_train = train[cols]; X_test = test[cols]
```

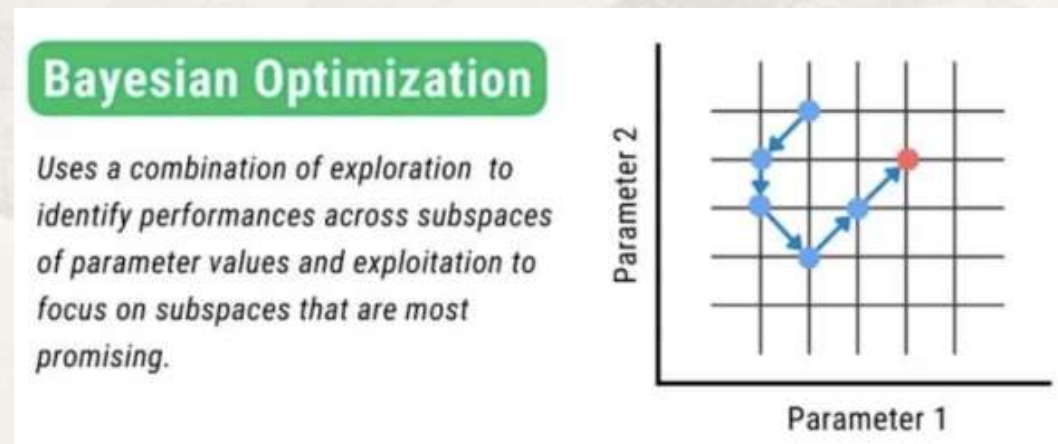
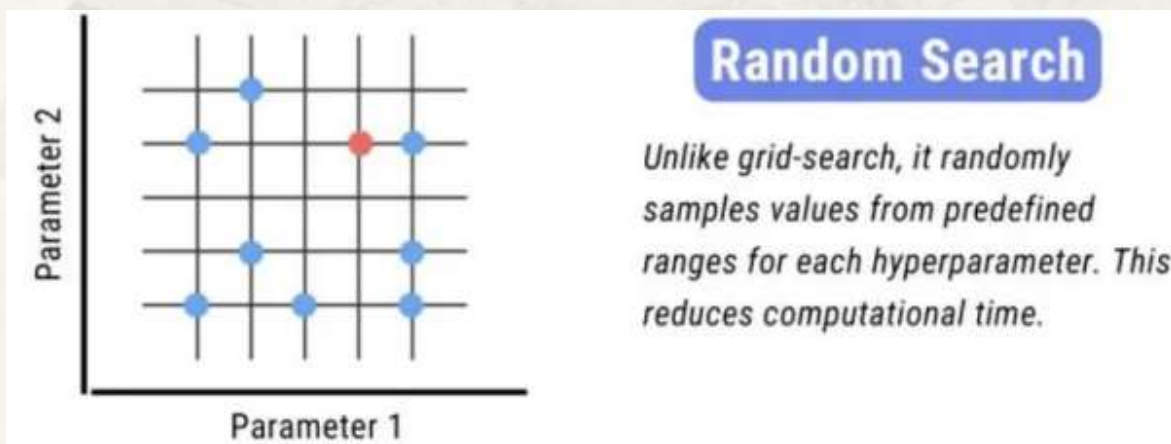
```
['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'casualt-1', 'casualt-2', 'casualt-3', 'casualt-4', 'casualt-5', 'weathert-1', 'weathert-2', 'weathert-3', 'weathert-4', 'weathert-5', 'tempt-1', 'tempt-2', 'tempt-3', 'tempt-4', 'tempt-5', 'atempt-1', 'atempt-2', 'atempt-3', 'atempt-4', 'atempt-5', 'humidityt-1', 'humidityt-2', 'humidityt-3', 'humidityt-4', 'humidityt-5', 'windspeedt-1', 'windspeedt-2', 'windspeedt-3', 'windspeedt-4', 'windspeedt-5']
```

```
Index(['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'casual', 'registered', 'count', 'date_time', 'casualt-1', 'casualt-2', 'casualt-3', 'casualt-4', 'casualt-5', 'weathert-1', 'weathert-2', 'weathert-3', 'weathert-4', 'weathert-5', 'tempt-1', 'tempt-2', 'tempt-3', 'tempt-4', 'tempt-5', 'atempt-1', 'atempt-2', 'atempt-3', 'atempt-4', 'atempt-5', 'humidityt-1', 'humidityt-2', 'humidityt-3', 'humidityt-4', 'humidityt-5', 'windspeedt-1', 'windspeedt-2', 'windspeedt-3', 'windspeedt-4', 'windspeedt-5'], dtype='object')
```


- 案例二的分析：基于时间序列的GBDT
 - 避免过拟合和或欠拟合，进行模型的超参数调优
 - 涉及多个超参数，如：
 - 迭代次数、最大树深度(基学习器)、损失函数、输入变量随机子集，等
 - 多个超参数的组合调优，可采用的策略
 - 网格搜索、随机搜索、贝叶斯搜索等
- 网格搜索：暴力穷举，在给定超参数空间内尝试所有可能的参数组合，计算成本高
 - 需指定一个超参数的取值范围；选出使模型性能最佳的超参数组合



- 案例二的分析：基于时间序列的GBDT
 - 避免过拟和或欠拟合，进行模型的超参数调优
 - 多个超参数组合的调优，可采用的策略
 - 网格搜索、随机搜索、贝叶斯搜索等
- 随机搜索：在超参数空间中随机选择超参数组合，不穷举所有的组合
 - 不保证能找到全局最优解，但通过随机选择一些组合，可在较短时间内探索较大的超参数空间
- 贝叶斯搜索：直观讲，若当前参数组合表现较好，则尝试在附近的超参数范围搜索(探索)，并探索其他可能(开发)
 - 概率代理模型(高斯过程)
 - 采集函数(期望改进函数，EI)



- 案例二的分析：基于时间序列的GBDT
 - 避免过拟和或欠拟合，进行模型的超参数调优(随机搜索详见代码，还可利用optuna调参)

#参数的网格搜索策略

```
parameters = {'n_estimators': [50, 100, 200],  
              'max_depth': np.arange(2, 10),  
              'loss': ['ls', 'huber'],  
              'max_features': ['auto', 'sqrt', 'log2']  
            }
```

```
GBRT = ensemble.GradientBoostingRegressor(random_state = 123)
```

```
grid = GridSearchCV(estimator = GBRT, param_grid = parameters, cv = 5, scoring = 'neg_mean_absolute_error', n_jobs=-1)  
grid.fit(X_train, y_train)
```

```
print('最优超参数:%s'%grid.best_params_)
```

```
BestModel = grid.best_estimator_
```

ls: 最小平方损失

auto: max_features=n_features

Huber损失:

$$L(\theta) = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2, & \text{if } |y_i - \hat{y}_i| \leq \delta \\ \delta \cdot (|y_i - \hat{y}_i| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

- 问:
 - 你认为这样的调参有问题吗?

- Huber损失的特点
 - 结合了MSE和MAE的优点
 - 问: 有什么优点?
 - 通过一个可调参数 δ 控制对离群点的敏感程度
 - 小误差(一般是噪声或模型接近收敛)时采用平方损失MSE, 平滑二次函数可保证梯度随误差减小, 实现精细且稳定的收敛
 - 大误差(一般是离群点)时采用线性损失MAE, 绝对值函数限制梯度的最大值(恒为 $\pm\delta$), 避免单个大误差主导梯度更新, 提高对离群点的鲁棒性
 - 当数据中存在可能的离群点, 既不希望完全忽略(MAE)也不希望过度惩罚(MSE), Huber 损失是一个很好的选择

- 案例二的分析：基于时间序列的GBDT

- 避免过拟合和或欠拟合，进行模型的超参数调优(随机搜索详见代码，还可利用optuna调参)

```
13 tscv = TimeSeriesSplit(n_splits = 5) #前向链式交叉验证
14 grid = GridSearchCV(estimator = GBRT, param_grid = parameters, cv = tscv, scoring = 'neg_mean_absolute_error', n_jobs=-1)
15
16 grid.fit(X_train, y_train)
17
18 end_time = time.time()
19 run_time = end_time - start_time
20 print('GBDT参数的网格搜索运行时间: %f秒' % run_time)
21
22 print('最优超参数: %s' % grid.best_params_)
23 BestModel = grid.best_estimator_

GBDT参数的网格搜索运行时间: 470.344782秒
最优超参数: {'loss': 'ls', 'max_depth': 4, 'max_features': 'auto', 'n_estimators': 100}
```

- TimeSeriesSplit: 前向链式交叉验证

- 逐渐扩大训练集，且让验证集始终位于训练集之后
- 验证集大小：参数test_size，默认N/n_splits取整数
 - 示例：

折	训练集索引	验证集索引
1	[0, 9] (10个)	[10, 19] (10个)
2	[0, 19] (20个)	[20, 29] (10个)
3	[0, 29] (30个)	[30, 39] (10个)
4	[0, 39] (40个)	[40, 49] (10个)

- 案例二的分析：基于时间序列的GBDT

- 模型评价和对比

```
1 yhat_train = BestModel.predict(X_train)
2 yhat_test = BestModel.predict(X_test)
3 print('训练误差(MAE): ', mean_absolute_error(y_train, yhat_train))
4 print('测试误差(MAE): ', mean_absolute_error(y_test, yhat_test))
```

训练误差(MAE): 7.070416319090313
测试误差(MAE): 10.120549732660882

不考虑时序特征的GBDT:

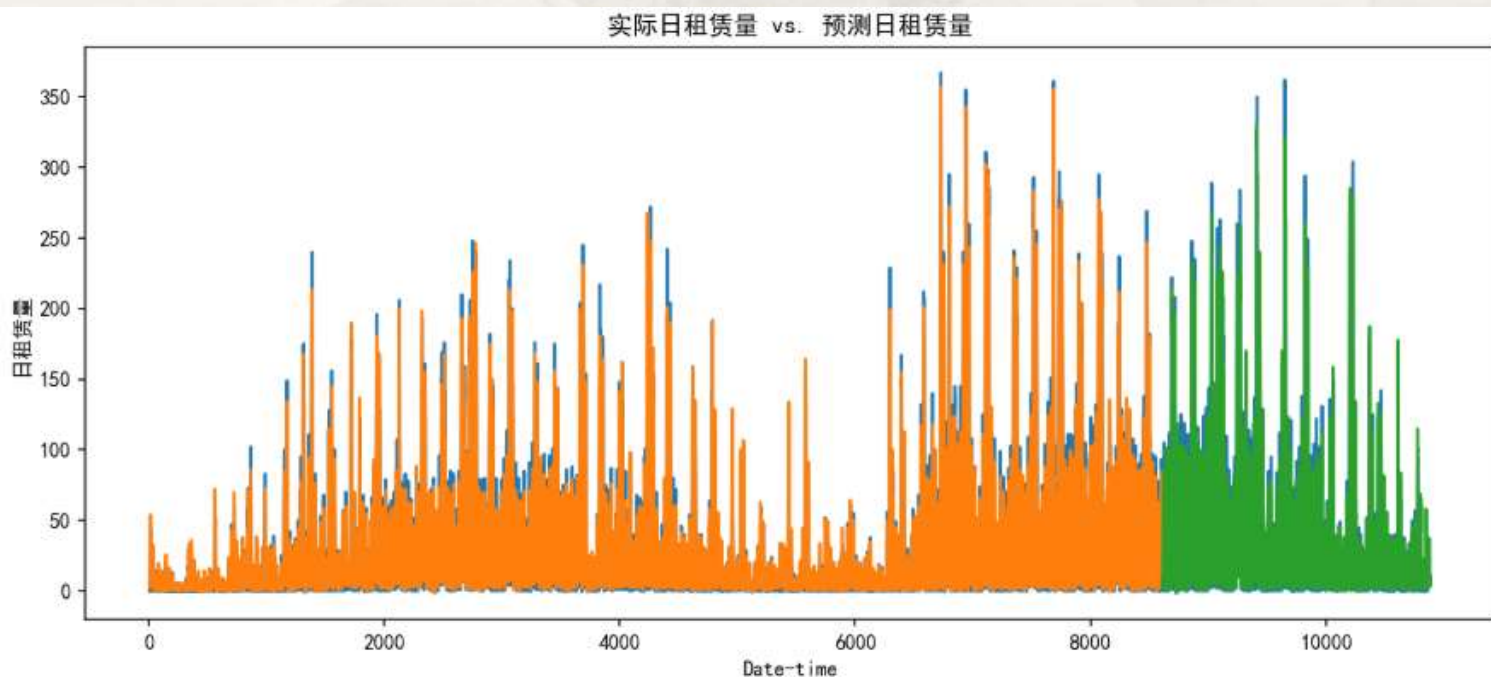
测试误差(MAE): GBDT(基学习器的最大树深度=2)=21.407
测试误差(MAE): GBDT(基学习器的最大树深度=7)=18.528

测试误差(MAE):

单棵决策树=19.961
Bagging=18.928
随机森林=18.011

- 案例二的分析：基于时间序列的GBDT
 - 可视化预测结果

```
1 train['y_pred_train'] = yhat_train # 保存训练集的预测值
2 test['y_pred_test'] = yhat_test # 保存测试集的预测值
3 actual_pred = pd.concat([train, test], axis=0) # 训练期和测试期纵向头尾连接
4 plt.figure(figsize = (12,5),dpi = 150)
5 plt.plot(actual_pred[['y', 'y_pred_train', 'y_pred_test']])
6 # 蓝色：实际值； 橙色：训练期的预测值； 绿色：测试期的预测值
7 plt.title('实际日租赁量 vs. 预测日租赁量')
8 plt.xlabel('Date-time')
9 plt.ylabel('日租赁量')
10 plt.show()
```



- 注意：
 - 回看(lookback)长度是个重要参数
- 课后练习：
 - 对registered的日租赁量进行预测，并对不同模型进行对比

• 问：GBDT如何应用在分类预测中？

- 回归预测中， b 次迭代是对 $b - 1$ 次迭代后残差的拟合

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \Delta f(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) - \rho_b \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

• 分类预测中的损失函数和 $f(\mathbf{X}_i)$

- 二元交叉熵损失 (Binary Cross-Entropy Loss), $y \in \{0, +1\}$, 也称对数损失 (Log Loss)

$$L(y_i, f(\mathbf{X}_i)) = -y_i f(\mathbf{X}_i) + \log(1 + \exp(f(\mathbf{X}_i)))$$

- 损失函数关于 $f(\mathbf{X}_i)$ 的梯度： $P(y_i = 1) - y_i$ 。伪响应变量： $\tilde{y}_i = y_i - P(y_i = 1)$

- 对数损失 ($y \in \{-1, +1\}$) 的等价形式：

$$L(y_i, f(\mathbf{X}_i)) = \log(1 + \exp(-2y_i f(\mathbf{X}_i))) \quad f(\mathbf{X}_i) = \frac{1}{2} \log \left(\frac{P(y_i = 1)}{P(y_i = -1)} \right)$$

$$P(y = 1 | \mathbf{X}) = \frac{1}{1 + \exp(-2f(\mathbf{X}))}$$

$$P(y = -1 | \mathbf{X}) = \frac{1}{1 + \exp(2f(\mathbf{X}))}$$

- 损失函数关于 $f(\mathbf{X}_i)$ 的梯度： $-2y_i P(y = -y_i)$ 。伪响应变量： $\tilde{y}_i = 2y_i P(y = -y_i)$
 - 仅取决于类别概率与类别值的差异

- 问：GBDT有哪些可以改进的方面？

- 计算效率方面

```
1 import time
2 start_time = time.time()
3
4 BestModel = ensemble.GradientBoostingRegressor(loss = 'ls', max_depth = 4, max_features='auto',
5                                                  n_estimators = 100, random_state = 123)
6 BestModel.fit(X_train,y_train)
7
8 end_time = time.time()
9 run_time = end_time - start_time
10 print('GBDT运行时间: %f秒' % run_time)
```

GBDT运行时间: 3.497642秒

- 问：GBDT有哪些可以改进的方面？

- 计算效率方面

- b 次迭代是对 $b - 1$ 次迭代后残差的拟合

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \Delta f(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) - \rho_b \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \beta_b h(\mathbf{X}_i; \gamma_b)$$

- 需进一步考虑残差的变化吗？

- 引进损失函数的二阶导

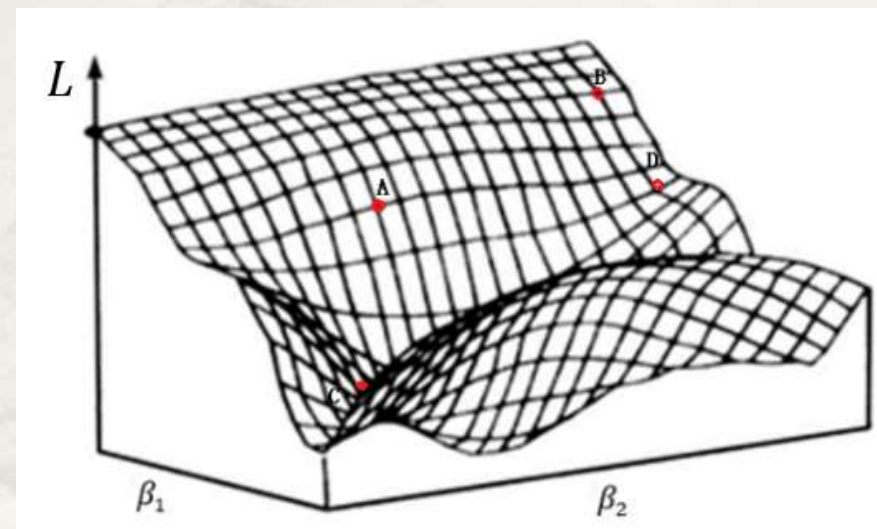
- 残差变化率高应怎样？残差变化率低应怎样？

- 优化确定最佳分组变量和组限的策略

- 泛化能力方面

- 最小化损失函数下的模型泛化能力怎样？

- 最小化损失函数+模型复杂度



弱模型到强模型：XGBoost算法

- XGBoost (eXtreme Gradient Boosting) 算法：属于梯度提升树

- 向前式分步可加建模方式的体现

$$f_b(\mathbf{X}) = f_{b-1}(\mathbf{X}) + \beta_b h(\mathbf{X}; \gamma_b)$$

$$h(\mathbf{X}; \gamma_b) = \sum_{j=1}^J b_{jb} I(\mathbf{X} \in R_{jb})$$

$$\sum_{j=1}^J b_{j1} I(\mathbf{X}_i \in R_{j1}) = b_{j1} + 0 + 0 + \dots = b_{j1}$$

- 基学习器为包含 J 个叶节点的决策树，第 b 次迭代后对样本观测 $\mathbf{X}_i$ 的联合预测结果：

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \sum_{j=1}^J \gamma_{jb} I(\mathbf{X}_i \in R_{jb}) = f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)$$

- Θ_{jb} 为 b 次迭代决策树叶节点 j 的预测值, 称为预测得分
 - 为什么不称预测值?

- 特点：

- 第一，每次迭代均针对目标函数, 使目标函数最小
 - 第二，通过泰勒展开得到损失函数在 $f_b(\mathbf{X})$ 处的近似表达
 - 第三，依据结构分数最小求解决策树 (优化决策树的分支策略, 不再直接基于GINI系数或熵)

弱模型到强模型：XGBoost算法

- XGBoost算法：目标函数
 - 构造目标函数，求得目标函数最小下的决策树
 - 对第 **b** 次迭代，目标函数为：损失函数+复杂度函数

$$obj_b(\Theta) = \sum_{i=1}^N L(y_i, f_b(\mathbf{X}_i)) + \Omega(f_b(\mathbf{X}))$$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \sum_{j=1}^J \gamma_{jb} I(\mathbf{X}_i \in R_{jb}) = f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)$$

$$obj_b(\Theta) = \sum_{i=1}^N L(y_i, f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)) + \Omega(f_{b-1}(\mathbf{X})) + \Omega(\Theta_b(\mathbf{X}))$$

- $\Omega(f_b(\mathbf{X}))$ 表示第 **b** 次迭代时组合模型(决策树)的复杂度
- $\Omega(\Theta_b(\mathbf{X}))$ 表示第 **b** 次迭代新增模型的复杂度

$$obj_b(\Theta) = \sum_{i=1}^N L(y_i, f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)) + \Omega(f_{b-1}(\mathbf{X})) + \Omega(\Theta_b(\mathbf{X}))$$

- XGBoost算法：目标函数的近似表达

- 通过泰勒展开，得到 **b** 次迭代结束后损失函数的近似，得到目标函数的近似表达

- 设有函数 $F(x)$ 在 x_0 处可导且高阶导数存在，泰勒展开：

- $$F(x_0 + \Delta x) = F(x_0) + \left[\frac{\partial F(x)}{\partial x}\right]_{x=x_0}(\Delta x) + \left[\frac{\partial^2 F(x)}{\partial x^2}\right]_{x=x_0} \frac{(\Delta x)^2}{2!} + \dots$$

- 对于样本观测 $\mathbf{X}_i$ ：

- $F(x)$ 对应损失函数 $L(y_i, f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i))$

- x_0 对应 $f_{b-1}(\mathbf{X}_i)$ ， Δx 对应 $\Theta_{jb}(\mathbf{X}_i)$

- b 次迭代结束时损失函数的近似为：

$$L(y_i, f_{b-1}(\mathbf{X}_i)) + g_i \Theta_{jb}(\mathbf{X}_i) + \frac{1}{2} h_i [\Theta_{jb}(\mathbf{X}_i)]^2$$

$$g_i = \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

$$h_i = \left[\frac{\partial^2 L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)^2} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

负残差的变化率

$$obj_b(\Theta) = \sum_{i=1}^N L(y_i, f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)) + \Omega(f_{b-1}(\mathbf{X})) + \Omega(\Theta_b(\mathbf{X}))$$

$$obj_b(\Theta) \approx \sum_{i=1}^N L(y_i, f_{b-1}(\mathbf{X}_i)) + g_i \Theta_{jb}(\mathbf{X}_i) + \frac{1}{2} h_i [\Theta_{jb}(\mathbf{X}_i)]^2 + \Omega(f_{b-1}(\mathbf{X})) + \Omega(\Theta_b(\mathbf{X}))$$

$$obj_b(\Theta) \approx \sum_{i=1}^N \left(g_i \Theta_{jb}(\mathbf{X}_i) + \frac{1}{2} h_i [\Theta_{jb}(\mathbf{X}_i)]^2 \right) + \Omega(\Theta_b(\mathbf{X}))$$

- 第**b**次迭代新增模型的复杂度：

$$\Omega(\Theta_b(\mathbf{X})) = \omega J + \frac{1}{2} \lambda \sum_{j=1}^J \Theta_{jb}^2$$

- J 为叶节点个数； ω 为复杂度系数； λ 为收缩参数
- 第二项还可利于目标函数的后续化简，且可将复杂度简化为 ωJ

- 第**b**次迭代目标函数的近似表达：

$$obj_b(\Theta) \approx \sum_{i=1}^N \left(g_i \Theta_{jb}(\mathbf{X}_i) + \frac{1}{2} h_i [\Theta_{jb}(\mathbf{X}_i)]^2 \right) + \omega J + \frac{1}{2} \lambda \sum_{j=1}^J \Theta_{jb}^2$$

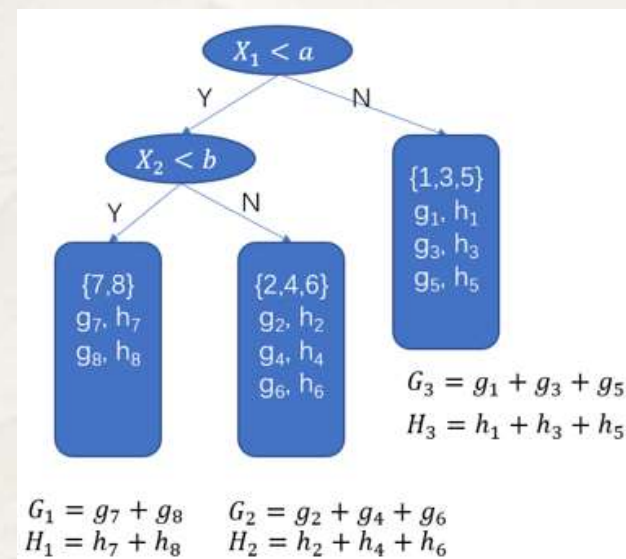
- 第***b***次迭代目标函数的近似表达：

$$obj_b(\Theta) \approx \sum_{i=1}^N \left(g_i \Theta_{jb}(\mathbf{x}_i) + \frac{1}{2} h_i [\Theta_{jb}(\mathbf{x}_i)]^2 \right) + \omega J + \frac{1}{2} \lambda \sum_{j=1}^J \Theta_{jb}^2$$

$$= \sum_{j=1}^J \left[\Theta_{jb} \sum_{\mathbf{x}_i \in \{j\}} g_i + \Theta_{jb}^2 \frac{1}{2} \left(\sum_{\mathbf{x}_i \in \{j\}} h_i + \lambda \right) \right] + \omega J$$

$$G_j = \sum_{\mathbf{x}_i \in \{j\}} g_i \quad H_j = \sum_{\mathbf{x}_i \in \{j\}} h_i$$

$$obj_b(\Theta) \approx \sum_{j=1}^J \left[G_j \Theta_{jb} + \frac{1}{2} (H_j + \lambda) \Theta_{jb}^2 \right] + \omega J$$



- XGBoost算法：决策树的求解
$$obj_b(\Theta) \approx \sum_{j=1}^J \left[G_j \Theta_{jb} + \frac{1}{2} (H_j + \lambda) \Theta_{jb}^2 \right] + \omega J$$

- G_j 和 H_j 已知，是关于 Θ_{jb} 的二次函数(存在最小值)
- 对 Θ_{jb} 求导并令导数等于0，得到第 b 次迭代叶节点 j 的预测得分：

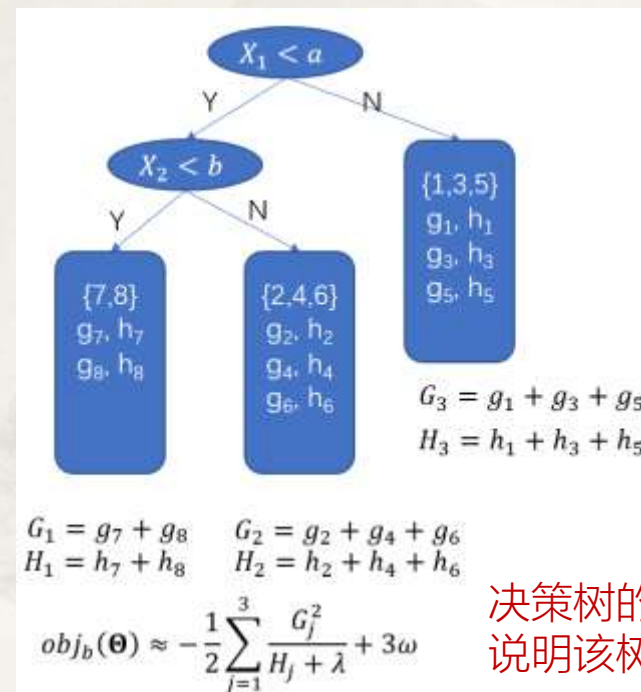
$$\Theta_{jb} = -\frac{G_j}{H_j + \lambda}$$

- $\Theta_{jb} = -\frac{G_j}{H_j + \lambda}$ 时，目标函数最小且等于：

$$obj_b(\Theta) \approx -\frac{1}{2} \sum_{j=1}^J \frac{G_j^2}{H_j + \lambda} + \omega J$$

- 称为新增决策树的**结构分数**

- 叶节点 j 的结构分数为：
$$-\frac{1}{2} \frac{G_j^2}{H_j + \lambda} + \omega$$

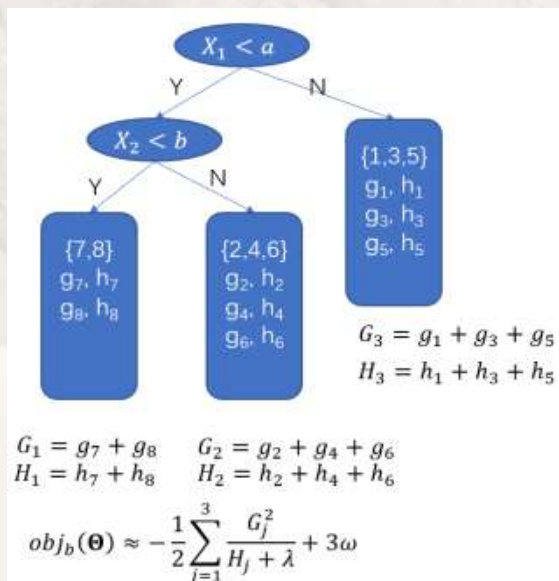


- XGBoost通常用以下标准确定当前最佳分组变量和组限值：

父节点j的结构分数： $-\frac{1}{2} \frac{G_j^2}{H_j + \lambda} + \omega$ 两子节点结构分数之和： $\left(-\frac{1}{2} \frac{G_{left}^2}{H_{left} + \lambda} + \omega\right) + \left(-\frac{1}{2} \frac{G_{right}^2}{H_{right} + \lambda} + \omega\right)$

父节点与子节点结构分数之差： $L_{split} = \frac{1}{2} \left[\frac{G_{left}^2}{H_{left} + \lambda} + \frac{G_{right}^2}{H_{right} + \lambda} - \frac{G_j^2}{H_j + \lambda} \right] - \omega$

- L_{split} 度量相对于父节点来说，其子节点带来的目标函数的减少程度，该值越大越好
- L_{split} 应至少大于某个阈值



Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I , instance set of current node

Input: d , feature dimension

score $\leftarrow 0$

$G \leftarrow \sum_{i \in I} g_i$, $H \leftarrow \sum_{i \in I} h_i$ 父节的G和H

for $k = 1$ to d do

$G_L \leftarrow 0$, $H_L \leftarrow 0$

 for j in sorted(I , by x_{kj}) do

$G_L \leftarrow G_L + g_j$, $H_L \leftarrow H_L + h_j$

$G_R \leftarrow G - G_L$, $H_R \leftarrow H - H_L$

 score $\leftarrow \max(\text{score}, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$

 end

end

Output: Split with max score

- 问：

- XGBoost引入二阶导的**意义**是什么？
 - 与GBDT的对比

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \Delta f(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) - \rho_b \left[\frac{\partial L(y_i, f(\mathbf{X}_i))}{\partial f(\mathbf{X}_i)} \right]_{f(\mathbf{X}_i)=f_{b-1}(\mathbf{X}_i)}$$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \beta_b h(\mathbf{X}_i; \gamma_b)$$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \sum_{j=1}^J \gamma_{jb} I(\mathbf{X}_i \in R_{jb}) = f_{b-1}(\mathbf{X}_i) + \Theta_{jb}(\mathbf{X}_i)$$

$$\Theta_{jb} = -\frac{G_j}{H_j + \lambda} = \frac{-G_j}{H_j + \lambda}$$

- 案例二的分析：基于时间序列的GBDT→XGBoost

```
start_time = time.time()
B = 100
modelXGB = xgb.XGBRegressor(max_depth = 3, n_estimators = B, random_state = 123)
modelXGB.fit(X_train, y_train, eval_set= [[X_test, y_test]], eval_metric = 'mae')
end_time = time.time()
run_time = end_time - start_time
print('XGB运行时间: %f秒' % run_time)
ErrXGB = modelXGB.evals_result()
```

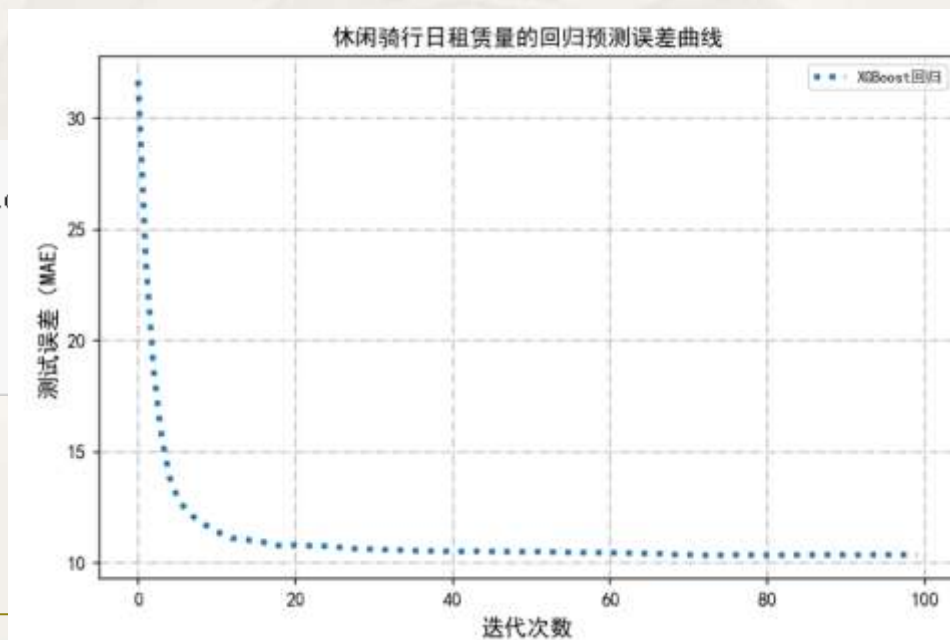
pip install xgboost

```
[99]      validation_0-mae:10.35882
XGB运行时间: 1.848645秒
```

- 计算效率提升，测试误差与GBDT基本持平

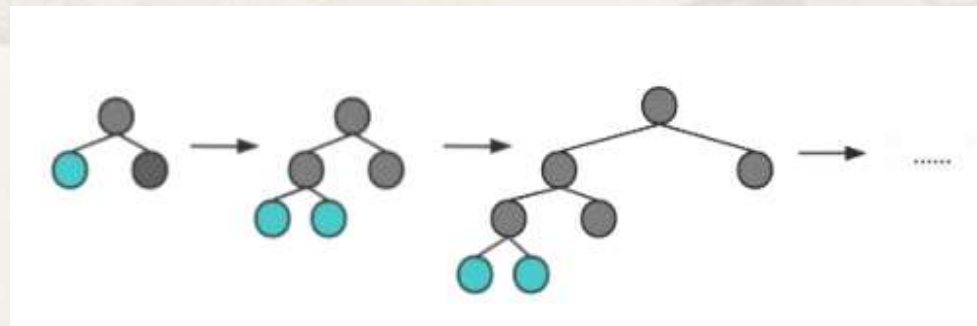
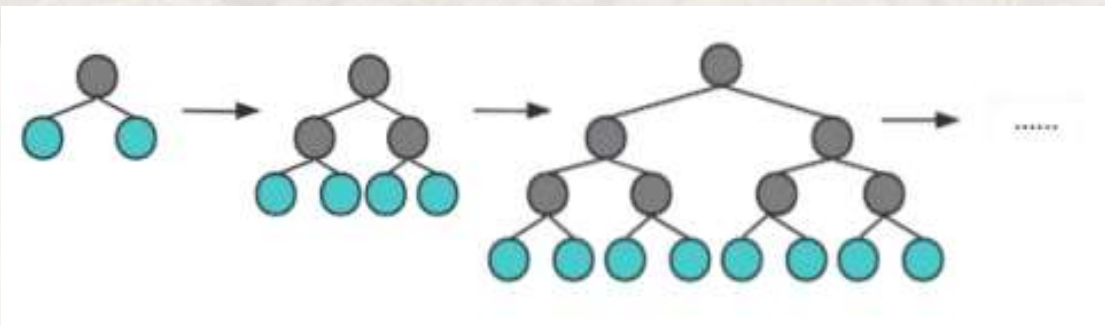
```
plt.figure(figsize = (8,5), dpi = 150)
plt.plot(np.arange(B), ErrXGB['validation_0']['mae'], linestyle = ':', label = 'XGBoost回归', linewidth = 2)
plt.title('休闲骑行日租赁量的回归预测误差曲线', fontsize = 12)
plt.grid(True, linestyle = '-.')
plt.xlabel('迭代次数', fontsize = 12)
plt.ylabel('测试误差 (MAE)', fontsize = 12)
plt.legend(fontsize = 8)
```

- 可进一步尝试超参数调优！



其他集成学习算法

- LightGBM(Light Gradient Boosting Machine, 轻量级梯度提升机): 提升算法效率
 - 采用基于直方图的决策树算法
 - 将连续的特征值离散化成直方图的桶, 在寻找最佳组限值时只需遍历桶即可, 从而降低计算复杂度
 - 引入梯度单边采样(Gradient-based One-Side Sampling, GOSS)技术, 在保持数据分布特征的同时减少数据在训练过程中的使用量
 - GOSS保留那些具有较大梯度(即错误较大)的数据样本, 对具有较小梯度的样本进行随机采样, 减少计算量且不会对模型的学习效果产生较大影响
 - “叶子优先(leaf-wise)”策略
 - 策略选择分组带来最大增益的叶子, 而不是在每一层上均匀分组, 提高计算效率



其他集成学习算法

- 案例二的分析：基于时间序列的GBDT→XGBoost→LightGBM

```
1 #pip install lightgbm
2 import lightgbm as lgb
3 lgb_params = {'objective': 'regression',      #默认lambda_l1 = 0.0; lambda_l2=0
4               'metric': {'mae'},            # 损失
5               'max_depth': 5,                # 最大树深度
6               'num_leaves': 10,              # 最大叶子数
7               'learning_rate': 0.02,         # 学习率
8               'feature_fraction': 0.8,       # 随机选取 80% 的特征到
9               'early_stopping_rounds': 50,    # 如果精度没有提高, 就停止训练
10              'n_estimators': 200,           # 迭代次数, 即最大树棵数
11              'nthread': -1,                 # 使用计算机上的所有内核
12              'verbose': 1                   # 训练进度 (0: 不显示任何内容)
13      }
14
15 # Use the Dataset class of lightGBM
16 lgbtrain = lgb.Dataset(data=X_train, label = y_train)
17 lgbtest = lgb.Dataset(data=X_test, label = y_test)
18 model = lgb.train(lgb_params, train_set=lgbtrain, valid_sets=[lgbtest]) #num_boost_round=200,
```

指定LightGBM的训练参数

以LightGBM的Dataset类形式组织数据

训练模型, 指定测试集作为评价依据

其他集成学习算法

- catBoost(Categorical Boosting): 主流GBDT框架的重要代表
 - 对分类型特征处理进行了优化, 特别适合有大量分类型变量的建模场景
 - 采用基于序列统计的目标编码策略 (以前讲过)
 - 基于排列的梯度估计机制
 - 传统GBDT每轮迭代均基于整个训练集估计梯度, 在类别不平衡等时易导致偏差
 - 改进: 对训练样本做多次独立的随机排列, 在每次迭代中仅使用当前样本之前的信息估计梯度
 - 引入对称树结构, 即每层节点使用相同的特征进行分组, 简化模型结构和训练效率

```
#pip install catboost
from catboost import CatBoostRegressor
model = CatBoostRegressor(
    iterations=200,          # 迭代次数, 即最大树棵数
    learning_rate=0.1,      # 学习率
    depth=5,                # 树的深度
    loss_function='mae',    # 损失函数,
    verbose=50,             # 每50轮输出一次训练日志
    early_stopping_rounds=50, # 如果验证集性能50轮不提升则提前停止
    random_seed=123         # 随机种子, 确保结果可重现
)
model.fit(X_train, y_train, eval_set=(X_test, y_test))
```